



The National University of Computer and Emerging Sciences
Peshawar Campus

Inheritance

CS1004 OBJECT ORIENTED PROGRAMMING

Muhammad Zulqarnain

Department of Computer Science



Inheritance

- Probably **most powerful feature** of **OOP**
- Concept **similar to inheritance** in **real life**
- **New classes** are **created** from **existing classes**



Introduction - Inheritance

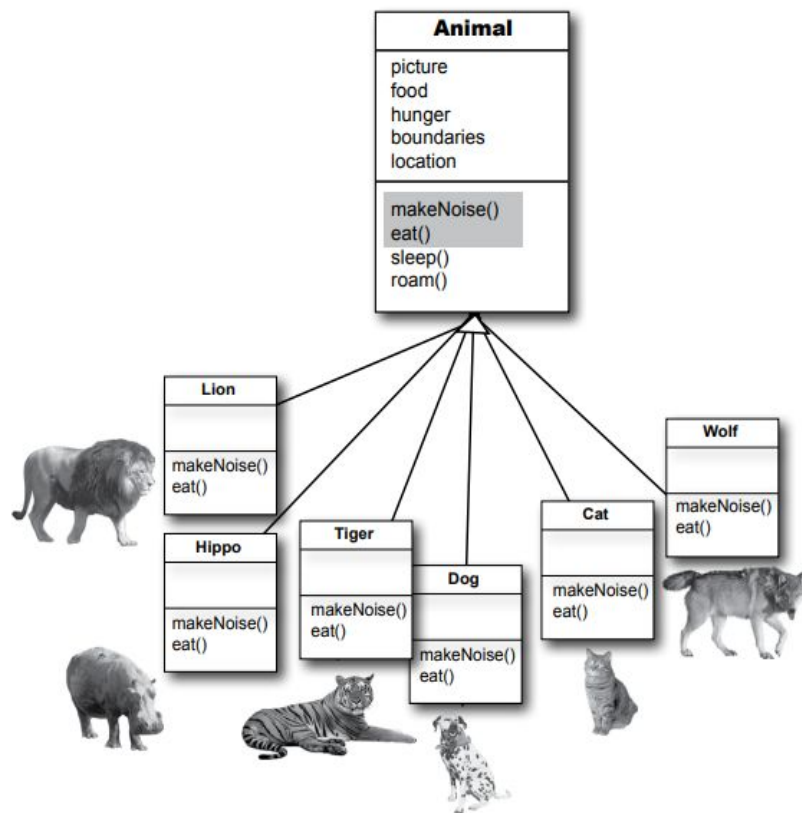
- **New classes absorb** all **features** of existing classes including their **data** and **functions**.
- Also **enhance** them by **adding** their **own new features** in form of **new data members** and **new member functions**



Is a relationship

Inheritance establishes an “**IS - A**” relationship between classes.

- Student is person
- Dog is animal
- Docter is person
- A poodle is a dog
- A car is a vehicle
- A flower is a plant
- A football player is an athlete
- Surgeon is docter and docter is a person(multilevel)





Introduction - Inheritance

- Existing classes are called **base classes**
- New classes are called **derived classes**
- Objects of derived classes are more **specialized** as compared to **objects of their base classes**

- **Inheritance** provides us a mechanism of **software reusability** which is one of the **most important principles** of **software engineering**



Inheriting Data and Functions

- All data members and member functions of base class are inherited to derived class
- Constructors, destructors and = operator are not inherited

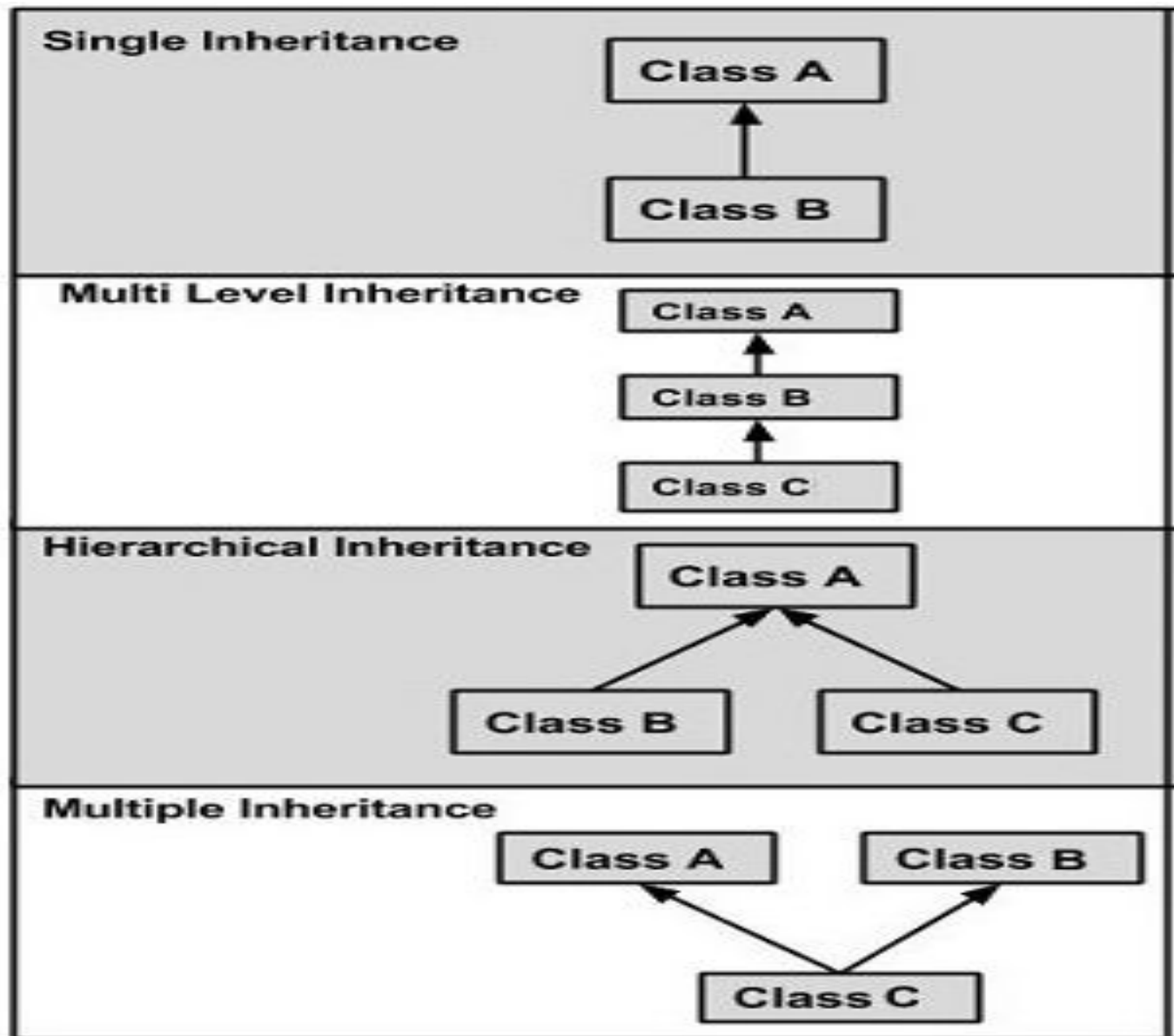


Some definitions in class hierarchy

- **Direct base class**
 - Inherited explicitly (one level up hierarchy)
- **Indirect base class**
 - Inherited two or more levels up hierarchy
- **Single inheritance**
 - Inherits from one base class
- **Multiple inheritance**
 - Inheritance from multiple classes



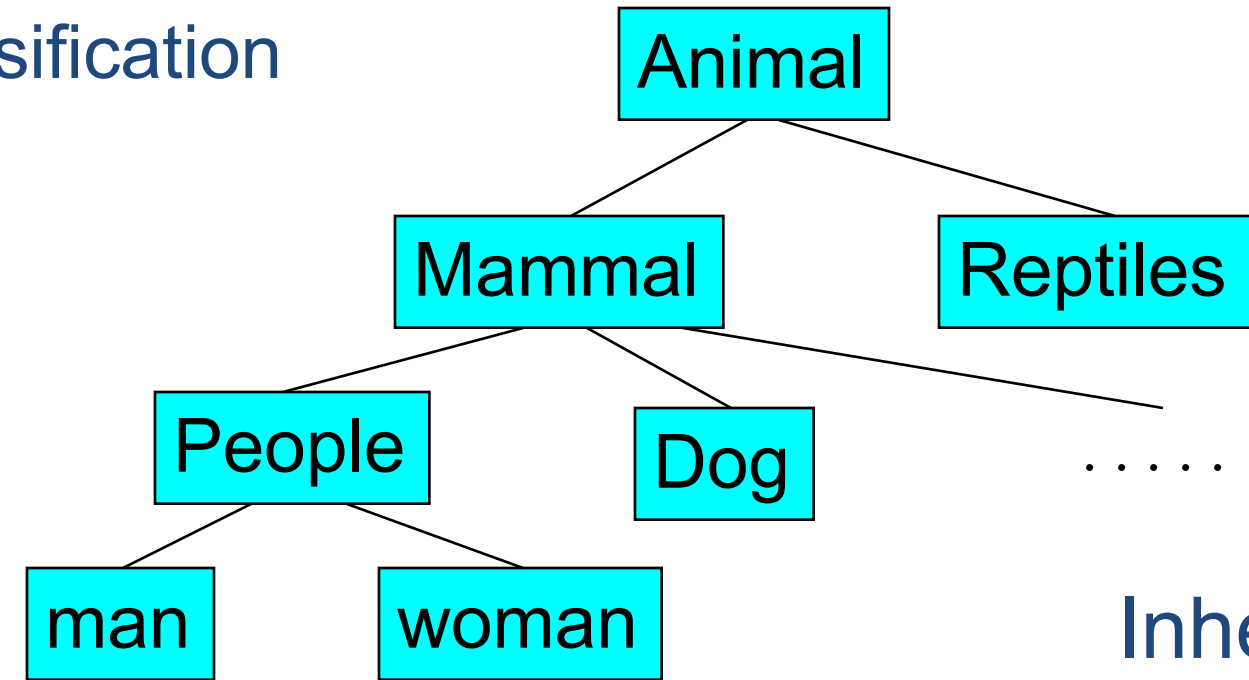
Types





Animals: Class's hierarchy

Classification



Inheritance

Joh
n

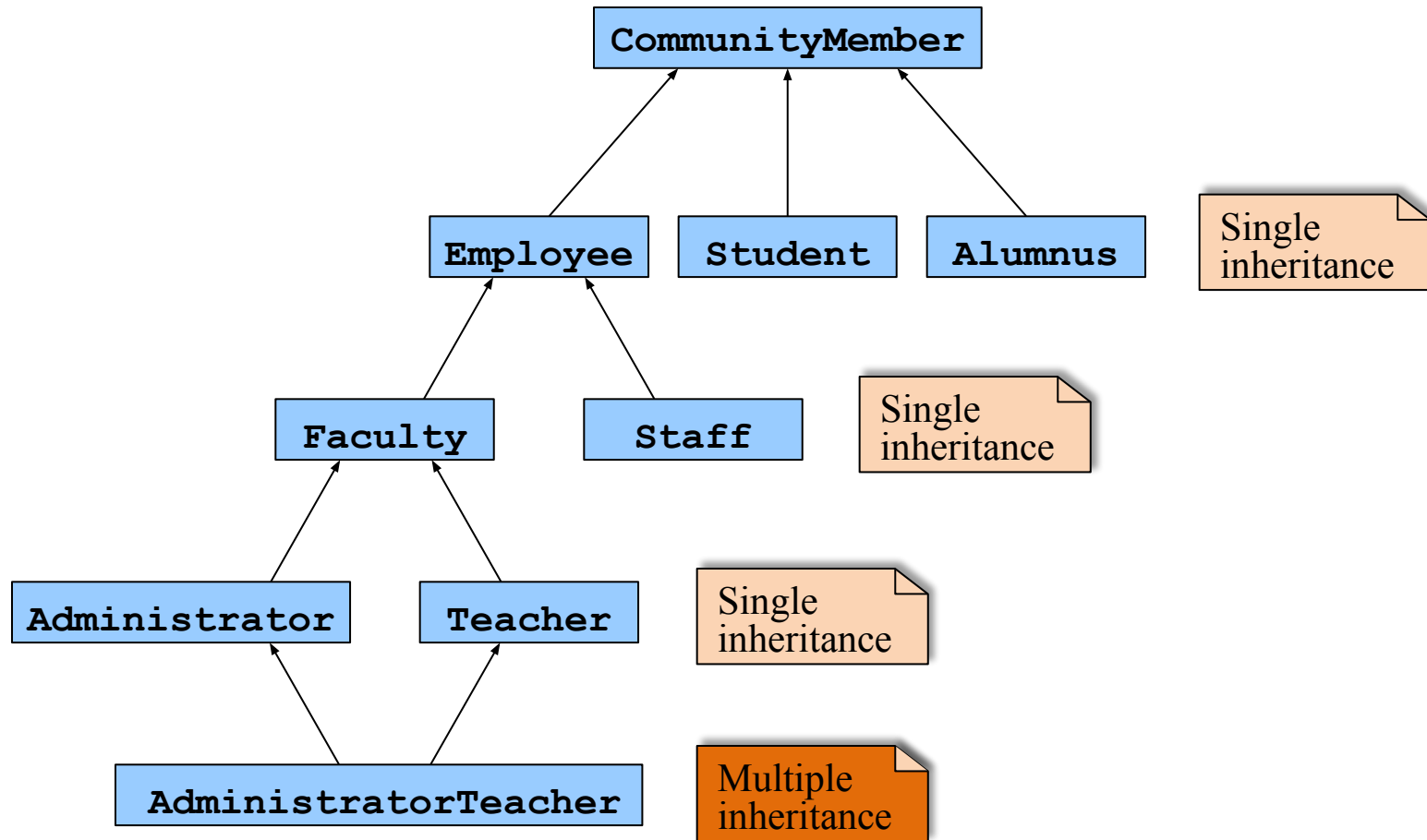
Mar
y



Inheritance Examples

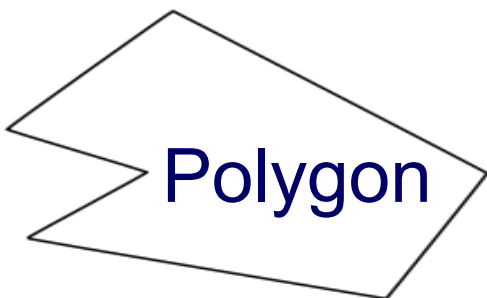
Base class	Derived classes
Student	GraduateStudent UndergraduateStudent
Shape	Circle Triangle Rectangle
Loan	CarLoan HomeImprovementLoan MortgageLoan
Employee	FacultyMember StaffMember
Account	CheckingAccount SavingsAccount

Another example: University's community member's hierarchy





Inheritance Concept



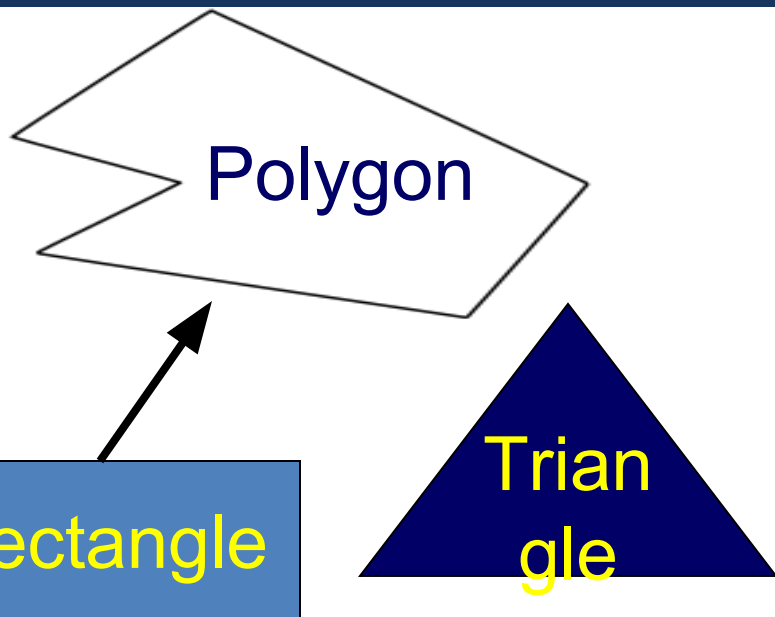
```
class Rectangle{  
    private:  
        int numVertices;  
        float xCoord, yCoord;  
    public:  
        void set(float x, float y, int nV);  
        float area();  
};
```

```
class Polygon{  
    private:  
        int numVertices;  
        float xCoord, yCoord;  
    public:  
        void set(float x, float y, int nV);  
};
```

```
class Triangle{  
    private:  
        int numVertices;  
        float xCoord, yCoord;  
    public:  
        void set(float x, float y, int nV);  
        float area();  
};
```



Inheritance Concept



```
class Polygon{  
    protected:  
        int numVertices;  
        float xCoord, float yCoord;  
    public:  
        void set(float x, float y, int nV);  
};
```

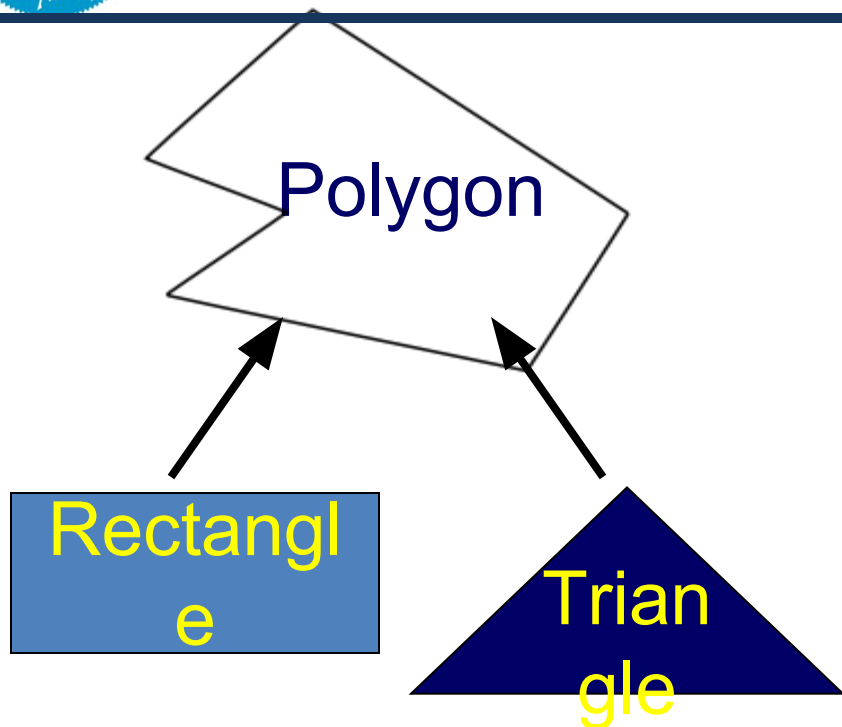
```
class Rectangle : public Polygon{  
    public:  
        float area();  
};
```



```
class Rectangle{  
    protected:  
        int numVertices;  
        float xCoord, float yCoord;  
    public:  
        void set(float x, float y, int nV);  
        float area();  
};
```



Inheritance Concept



```
class Triangle : public Polygon{
public:
    float area();
};
```

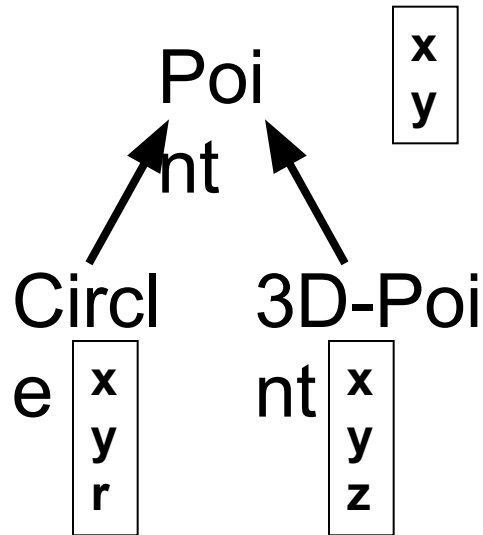


```
class Polygon{
protected:
    int numVertices;
    float xCoord, float yCoord;
public:
    void set(float x, float y, int nV);
};
```

```
class Triangle{
protected:
    int numVertices;
    float xCoord, float yCoord;
public:
    void set(float x, float y, int nV);
    float area();
};
```



Inheritance Concept



```
class Point{
protected:
    int x, y;
public:
    void set (int a, int b);
};
```

```
class Circle : public Point{
private:
    double r;
};
```

```
class 3D-Point: public Point{
private:
    int z;
};
```



Define a Class Hierarchy

- Syntax:

class *DerivedClassName* : **access-level** *BaseClassName*

where

- **access-level** specifies the **type of derivation**

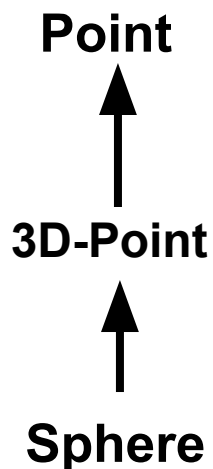
- **private** by default, or
- **public**

- **Any class can serve as a base class**

- Thus a **derived class** can also be a **base class**



Class Derivation



```
class 3D-Point : public Point{  
protected:  
    double z;  
    ... ..  
};
```

```
class Point{  
protected:  
    int x, y;  
public:  
    void set (int a, int b);  
};
```

```
class Sphere : public 3D-Point{  
private:  
    double r;  
    ... ..  
};
```

Point is the base class of **3D-Point**, while **3D-Point** is the base class of **Sphere**



What to inherit?

- **In principle**, every member of a **base class** is **inherited** by a **derived class**
 - just with different access permission

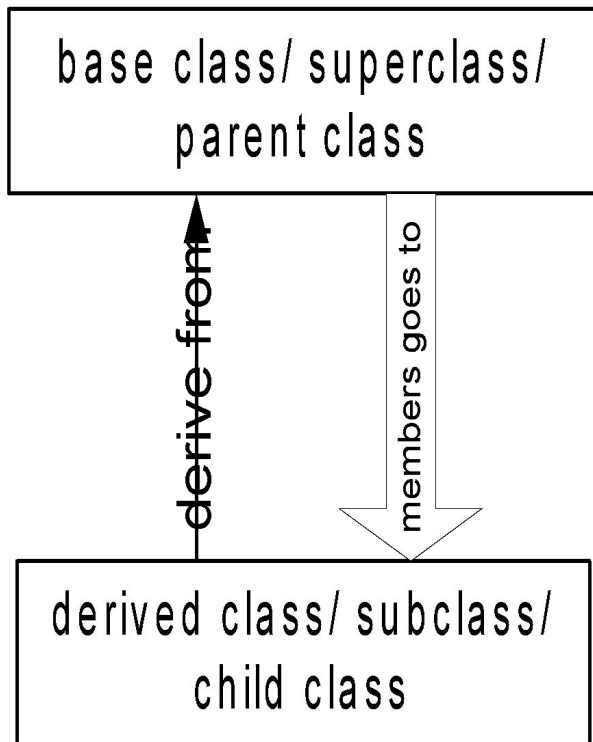


Access Control Over the Members

- Two levels of access control over class members

- **class definition**

- **inheritance type**



```
class Point{  
    protected: int x, y;  
    public: void set(int a, int b);  
};
```

```
class Circle : public Point{  
    ... ..  
};
```

Constructor Rules for Derived Classes

The **default constructor** and the **destructor** of the **base class** are always called when a new object of a **derived class** is created or destroyed.

```
class A {  
public:  
    A ( )  
        { cout<< "A:default"<<endl; }  
    A (int a)  
        { cout<<"A:parameter"<<endl; }  
};
```

```
class B : public A  
{  
public:  
    B (int a)  
        { cout<<"B"<<endl; }  
};
```

```
B test(1);
```

output:

```
A:default  
B
```



Constructor Rules for Derived Classes

You can also **specify** a **constructor** of the **base class** other than the **default constructor**

```
DerivedClassCon ( derivedClass args ) : BaseClassCon ( baseClass args )  
  
{ DerivedClass constructor body }
```

```
class A {  
    public:  
    A ( )  
        {cout<< "A:default"<<endl;}  
    A (int a)  
        {cout<<"A:parameter"<<endl;}  
};
```

```
class C : public A {  
    public:  
    C (int a) : A(a)  
        {cout<<"C"<<endl;}  
};
```

C test(1);

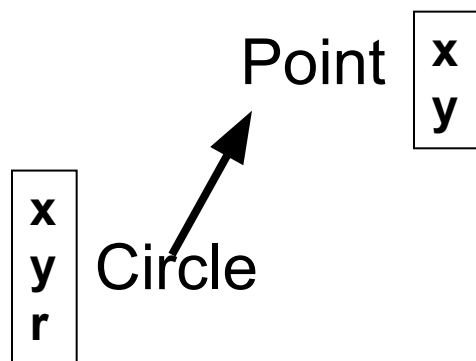
output
:

A:parameter
C



Define its Own Members

The derived class can also define its own members, in addition to the members inherited from the base class



```
class Circle : public Point{
private:
    double r;
public:
    void set_r(double c);
};
```

```
class Point{
protected:
    int x, y;
public:
    void set(int a, int b);
};
```

```
class Circle{
protected:
    int x, y;
private:
    double r;
public:
    void set(int a, int b);
    void set_r(double c);
};
```



Even more ...

- A **derived class** can **override** methods defined in its **parent class**. With overriding:
 - the **method** in the **subclass** has the **identical signature** to the **method** in the **base class**.
 - a **subclass implements** its **own version** of a **base class method**.

```
class A {  
protected:  
    int x, y;  
public:  
    void print ()  
    { cout<<"From A"<<endl; }  
};
```

```
class B : public A {  
    public:  
    void print ()  
        {cout<<"From B"<<endl;}  
};
```



Access a Method

```
class Point{  
protected:  
    int x, y;  
public:  
    void set(int a, int b)  
        { x=a; y=b; }  
    void foo ();  
    void print();  
};
```

Point A;
A.set(30,50); // from base class Point
A.print(); // from base class Point

```
class Circle : public Point{  
private:  double r;  
public:  
    void set (int a, int b, double c) {  
Point :: set(a, b); //same name function call  
        r = c;  
    }  
    void print();  
};
```

Circle C;
C.set(10,10,100); // from class Circle
C.foo (); // from base class Point
C.print(); // from class Circle



point and circle classes

```
class Point
{
    protected:
        int x,y;
    public:
        Point(int ,int);
        void display(void);
};

Point::Point(int a,int b)
{
    x=a;
    y=b;
}

void Point::display(void)
{
    cout<<"point = [" <<x<<" "<<y<<""]";
}
```



```
class Circle : public Point
{
    double radius;
public:
    Circle(int ,int ,double );
    void display(void);
};
```

```
Circle::Circle(int a,int b,double c):Point(a,b) {
    radius = c;
}
```

```
void Circle::display(void) {
    Point::display();
    cout<<" and radius = "<<radius;
}
```



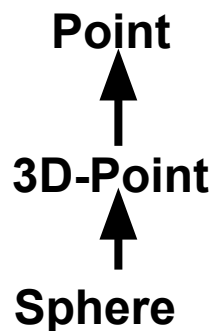
```
int main(void)
{
    Circle c(3,4,2.5);
    c.display();
    return 0;
}
```

Output:

point=[3,4] and radius = 2.5



Class Derivation



```
class Point{  
    protected:  
        int x, y;  
    public:  
        void set (int a, int b);  
};
```

```
class 3D-Point : public Point{  
    private:  
        double z;  
        ... ..  
};
```

```
class Sphere : public 3D-Point{  
    private:  
        double r;  
        ... ..  
};
```

Point is the **base class** of **3D-Point**, while **3D-Point** is the **base class** of **Sphere**

Order of execution of Constructors/Destructors

- **Chain** of **constructor calls**:
 - **Last constructor** called in chain
 - **First constructor body** to finish executing
 - Example:
 - Point**□**Circle**□**Cylinder** hierarchy
 - **Point constructor** body **finishes execution first**
 - Then **circle** and last **Cylinder**



Order of execution of Constructors/Destructors

- **Chain of destructor calls**
 - Reverse order of constructor chain
 - Destructor of derived-class called first
 - Destructor of next base class up hierarchy next
 - Continue up hierarchy until final base reached
 - After final base-class destructor, object removed from memory



Example

```
class Point
```

```
{  
    protected:  
        int x,y;  
    public:  
        Point(int ,int );  
        void display(void);  
        ~Point() { cout<<"\nPoint Class Destructor\n"; }  
};
```

```
Point::Point(int a,int b) {  
    cout<<"\nPoint Class Constructor\n";  
    x=a;  
    y=b;  
}
```

```
void Point::display(void) {  
    cout<<"point = [" <<x<<" "<<y<<" ]";  
}
```



Example – cont.

```
class Circle : public Point
{
    protected:
        double radius;
    public:
        Circle(int ,int ,double);
        void display(void);
        ~Circle() { cout<<"\nCircle Class Destructor\n"; }
};
```

```
Circle::Circle(int a,int b,double c):Point(a,b) {
    cout<<"\nCircle Class Constructor\n";
    radius = c;
}
```

```
void Circle::display(void) {
    Point::display();
    cout<<" radius = "<<radius;
}
```




Example – cont.

```
class Cylinder: public Circle
```

```
{  
    double height;  
    public:  
        Cylinder(int ,int ,double ,double);  
        void display(void);  
        double GetVolume(void);  
        ~Cylinder() { cout<<"\nCylinder Class Destructor\n"; }  
};
```

```
Cylinder::Cylinder(int a,int b,double r,double h):Circle(a,b,r) {  
    cout<<"\nCylinder Class Constructor\n";  
    height=h;  
}
```

```
double Cylinder::GetVolume(void) {  
    return 3.14 * radius * radius * radius;  
}
```

```
void Cylinder::display(void) {  
    Circle::display();  
    cout<<" height = "<<height;  
}
```



Example – cont.

```
int main(void)
{
    Cylinder c(3,4,2.5,3.7);
    return 0;
}
```

Output:

Point Class Constructor
Circle Class Constructor
Cylinder Class Constructor
Cylinder Class Destructor
Circle Class Destructor
Point Class Destructor



Types of inheritance

1. public
2. private
3. protected



Public Inheritance

- With **public inheritance**,
 - **public** and **protected** members of the **base class** become respectively public and protected members of the derived class.



Protected Inheritance

- **Public** and **protected members** of the **base class** become protected members of the **derived class**.



Private Inheritance

- With **private inheritance**, **public** and **protected** members of the **base class** become private members of the derived class.

public, protected and private Inheritance

Modifier	Access from same class	Access from subclass	Access from anywhere
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes



Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

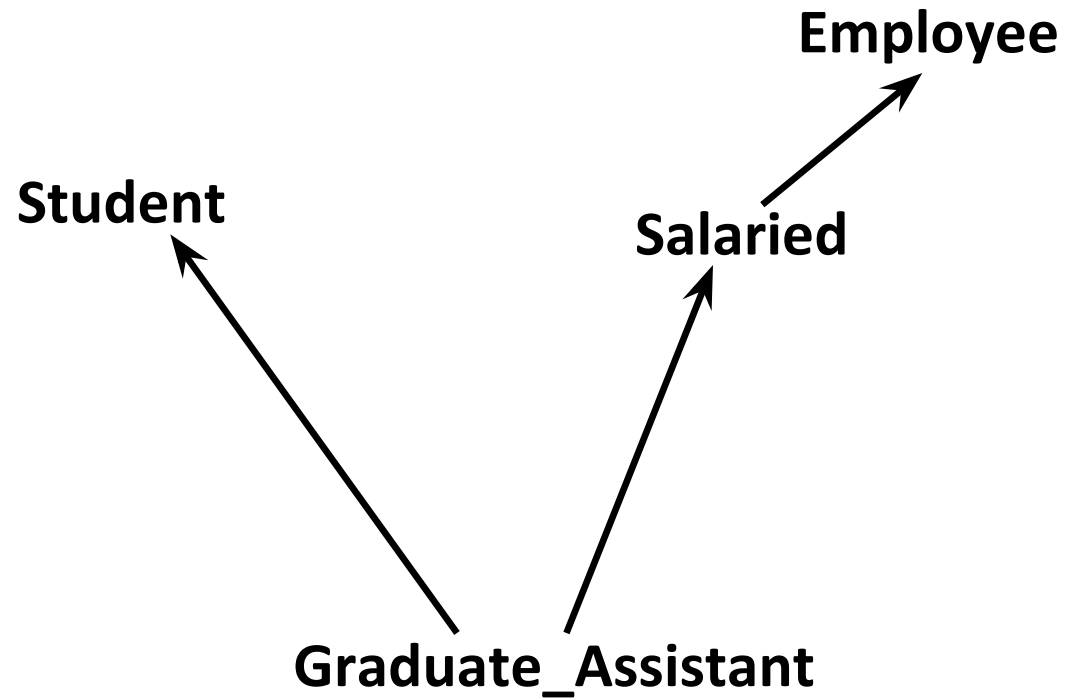
We will focus on the public derivation

public, protected and private Inheritance

Base class member access specifier	Type of inheritance		
	public inheritance	protected inheritance	private inheritance
Public	public in derived class. Can be accessed directly by any non- static member functions, friend functions and non-member functions.	protected in derived class. Can be accessed directly by all non- static member functions and friend functions.	private in derived class. Can be accessed directly by all non- static member functions and friend functions.
Protected	protected in derived class. Can be accessed directly by all non- static member functions and friend functions.	protected in derived class. Can be accessed directly by all non- static member functions and friend functions.	private in derived class. Can be accessed directly by all non- static member functions and friend functions.
Private	Hidden in derived class. Can be accessed by non- static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non- static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non- static member functions and friend functions through public or protected member functions of the base class.



Multiple Inheritance





What is Multiple Inheritance?

- If **class A inherits** from **more than one class**,
 - i.e., $A \sqsupseteq (B_1, B_2, \dots, B_n)$, we speak of **multiple inheritance**.
- This **may introduce naming conflicts**:
 - if **at least two** of its **super_classes** define properties (*data members* or *member functions*) with the **same name**.



//simple example showing multiple inheritance

```
class base1 {
public:
    void funbase1(void) { cout<<"funbase1"; }
};

class base2 {
public:
    void funbase2(void) { cout<<"funbase2"; }
};

class derived:public base1, public base2 {
public:
    void funderived(void) { cout<<"funderived"; }
};

void main(void) {
    derived der;
    der.funbase1();
    der.funbase2();
    der.funderived();
}
```



Example (ambiguity)

```
class A { public: void Foo() {} }  
class B : public A {}  
class C : public A {}  
class D : public B, public C {}
```

```
D d;  
d.Foo();
```

is this B's Foo() or C's Foo() ??



Solution (virtual inheritance)

```
class A {  
    public: void Foo() {}  
}
```

```
class B : public virtual A {  
}
```

```
class C : public virtual A {  
}
```

```
class D : public B, public C {  
}
```

```
D d;  
d.Foo(); // no longer ambiguous
```



Another example

```
class PoweredDevice
{
    public:
    PoweredDevice(int nPower)
    {
        cout << "PoweredDevice: " << nPower << endl;
    }
};
```



Another example

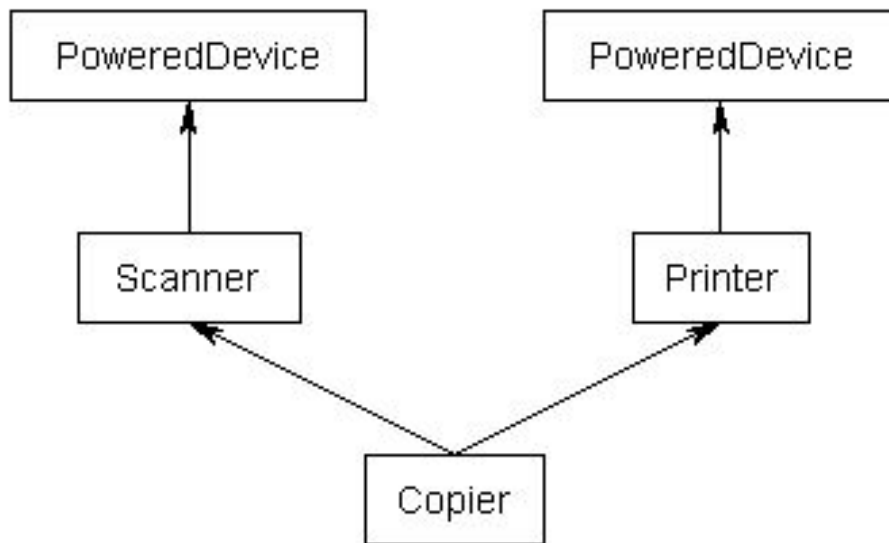
```
class Scanner: public PoweredDevice {
public:
    Scanner(int nScanner, int nPower) : PoweredDevice(nPower)
    {
        cout << "Scanner: " << nScanner << endl;
    }
};

class Printer: public PoweredDevice {
public:
    Printer(int nPrinter, int nPower) : PoweredDevice(nPower)
    {
        cout << "Printer: " << nPrinter << endl;
    }
};
```



Another example

```
class Copier: public Scanner, public Printer
{
    public:
    Copier(int nScanner, int nPrinter, int nPower) : Scanner(nScanner,
nPower), Printer(nPrinter, nPower)
    {
    }
};
```





Another example

```
int main()
{
    Copier cCopier(1, 2, 3);
}
```

What should be the output?

PoweredDevice: 3

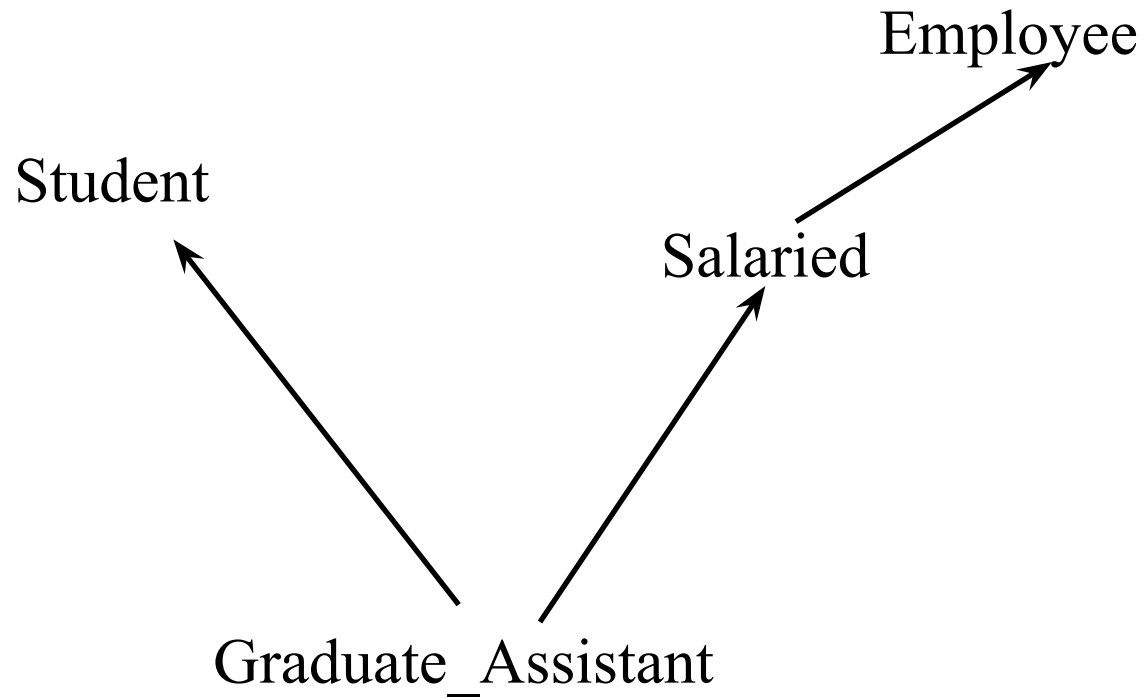
Scanner: 1

PoweredDevice: 3

Printer: 2



Multiple Inheritance





//Example (showing ambiguity)

```
typedef unsigned long ulong;
```

```
class Student {  
    ulong id;  
    unsigned age;  
public:  
    unsigned GetAge() const { return age; }  
    ulong GetId() const { return id; }  
    void SetAge( unsigned n ) { age = n; }  
    void SetId( ulong n ) { id=n; }  
};
```



```
class Employee {
```

```
public:
```

```
    unsigned GetAge() const { return age; }
```

```
    void SetAge( unsigned n ) { age = n; }
```

```
    void SetId( ulong n) { id=n; }
```

```
    ulong GetId(void) const { return id; }
```

```
private:
```

```
    unsigned age;
```

```
    ulong id;
```

```
};
```



```
class Salaried :public Employee {  
    float salary;  
public:  
    float GetSalary() const { return salary; }  
    void SetSalary( float s ) { salary=s; }  
};
```

```
class GradAssistant :public Student, public Salaried {  
public:  
    void Display() const;  
};
```

```
void GradAssistant::Display() const {  
    cout<<GetId()<<" "<<GetSalary()<<" "<<GetAge(); //ambiguity  
}
```



```
int main(void) {  
  
    GradAssistant ga;  
    ga.SetAge(20); //ambiguity  
    ga.SetId(15);  //ambiguity  
    ga.Display();  //ambiguity inside display function  
  
} //program will not compile and will generate errors
```



What is the solution?

- Call functions explicitly by specifying name of class and using scope resolution operator to remove ambiguity
 - Direct solution: `Student::SetAge()` or `Salaried::SetAge()`.
 - Virtual inheritance



//Example 2 corrected

```
typedef unsigned long ulong;
```

```
class Student {
```

```
    ulong id;
```

```
    unsigned age;
```

```
public:
```

```
    unsigned GetAge() const { return age; }
```

```
    ulong GetId() const { return id; }
```

```
    void SetAge( unsigned n ) { age = n; }
```

```
    void SetId( ulong n ) { id=n; }
```

```
};
```

```
class Employee {
```

```
public:
```

```
    unsigned GetAge() const { return age; }
```

```
    void SetAge( unsigned n ) { age = n; }
```

```
    void SetId( ulong n) { id=n; }
```

```
    ulong GetId(void) const { return id; }
```

```
private:
```

```
    unsigned age;
```

```
    ulong id;
```

```
};
```




```
class Salaried :public Employee {
    float salary;
public:
    float GetSalary() const { return salary; }
    void SetSalary( float s ) { salary=s; }
};

class GradAssistant :public Student, public Salaried {
public:
    void Display() const;
};

void GradAssistant::Display() const {
    cout<<Employee::GetId()<<" "<<Salaried::GetSalary()<<" "<<Salaried::GetAge();
}

void main(void) {
    GradAssistant ga;
    ga.Employee::SetAge(20);
    ga.Employee::SetId(15);
    ga.Salaried::SetSalary(2000);
    ga.Display();
}

//no ambiguity now, program will compile successfully now
```